Day 5: AI Agents 101 — MCP and Open-Source Inference

Study notes • Agent architecture, MCP, tool safety, model serving, and multi-agent systems

Note: These are newly prepared study notes aligned with your day-wise schedule, not pages extracted from the original AMD PDF.

1. Learning objectives

• Distinguish a simple chatbot from a tool-using assistant and an agent workflow.

• Describe model, tools, instructions, state, orchestration, and guardrails.

• Explain the basic concepts of Model Context Protocol (MCP).

• Outline how an open-weight model can be tested and served for inference.

2. What is an AI agent?

• An AI agent uses a model to interpret a goal, choose a next step, and interact with tools or an environment. It may observe results and continue until a stopping condition is reached.

• System | Typical behavior

• Basic chatbot | Responds mainly from the prompt and model knowledge.

• Tool-using assistant | Calls a tool and uses its returned result.

• Agent workflow | Can select and sequence actions, inspect outcomes, and continue toward a goal.

3. Core architecture and loop

• Component | Role

• Model | Interprets input and proposes responses or actions.

• Instructions / policy | Defines the goal, boundaries, and expected behavior.

• Tools | Perform operations such as search, calculation, or database queries.

• State / memory | Maintains relevant task context when appropriate.

• Orchestrator | Routes calls, manages the loop, and checks stopping conditions.

• Guardrails | Validate inputs, permissions, tool results, and outputs.

• Typical loop: receive the goal; check that it is clear and allowed; choose a tool or response; validate the call; execute and inspect the result; update task state; repeat if needed; return a grounded answer or request approval.

4. MCP: Model Context Protocol

• MCP is an open protocol designed to connect AI applications with external tools and data sources through a consistent interface.

• Host: the AI application coordinating the experience. Client: the component that communicates with an MCP server. Server: exposes capabilities to the client.

• Tools are callable operations. Resources provide data or context. Prompts can provide reusable prompt templates.

• Capabilities depend on the client and server implementation. Connecting to an MCP server does not automatically make its tools safe; permissions and validation remain necessary.

5. Safe tool use

• Use least privilege: expose only the tools and data needed for the task.

• Validate arguments and reject unexpected or malformed inputs.

• Treat retrieved text and tool output as data, not as trusted instructions.

• Ask for confirmation before irreversible actions such as deleting records, sending messages, or making purchases.

• Keep secrets out of prompts and logs. Use appropriate secret management and audit actions without collecting unnecessary personal data.

6. Open-source inference: deployment steps

• Choose a model based on quality, language support, context length, hardware needs, and license. Read the model card and license first.

• Prepare the runtime and dependencies, download model weights from a trusted source, and load the model and tokenizer.

• Configure device, numerical precision, context length, and generation settings.

• Run a simple test prompt and verify the output before exposing a service.

• For deployment, add authentication, rate limits, monitoring, error handling, and input/output safeguards.

7. Serving and optimization

• Quantization reduces numerical precision to lower memory use, often trading off some quality or speed depending on hardware.

• Batching processes multiple requests together to improve throughput, though it may affect latency.

• The KV cache stores attention-related values from previous tokens to avoid recomputing them during generation.

• Inference engines such as vLLM can improve throughput and memory management for supported models.

• Useful metrics include time to first token, tokens per second, concurrency, error rate, memory use, and output quality.

8. Multi-agent systems

• Multi-agent systems use multiple agents or role-specialized components that coordinate on a task.

• Sequential pipeline: one agent passes its output to the next. Coordinator-and-specialists: a coordinator delegates subtasks. Parallel exploration: several agents investigate independently. Critic-reviewer: one component checks another against explicit criteria.

• More agents do not automatically improve results. Coordination cost, conflicting outputs, duplicated work, and error propagation must be managed.

9. Deployment checklist

• Define task scope, success criteria, and permitted actions.

• Test each tool independently before connecting it to an agent.

• Handle timeouts, tool failures, malformed results, and retry limits.

• Set maximum steps, token budgets, and stopping conditions.

• Evaluate task success, tool-call accuracy, latency, cost, and safety.

• Provide a human escalation path when confidence is low or an action is consequential.

Practical Exercises

1. Draw an agent architecture for a campus assistant that answers questions using a timetable tool.

2. List three tools for an agent and define input validation and permissions for each.

3. Plan an open-model inference test recording model, hardware, latency, throughput, and output quality.

4. Design a human-approval step before an agent sends an email or changes a database record.

Review Questions

1. What distinguishes an agent workflow from a single model call?

2. What are the host, client, and server roles in MCP?
3. What is the difference between an MCP tool and a resource?
4. Why should tool calls be validated and permission-limited?
5. What trade-off can quantization introduce?
6. Which metrics help evaluate an inference service?
7. Give two benefits and two risks of multi-agent systems?